

# Java Section 4

## ARRAYS AND ARRAYLISTS

***You will learn to use arrays to store data  
from lists of values.***

# Java Section 4

## Arrays

---

- What is an array?
- How to declare, create and initialise arrays
- How to work with individual array elements
  
- The enhanced for statement.
- Arrays and Methods
  
- Class Arrays                      `java.util.Arrays`
- Class ArrayLists                `java.util.ArrayList`
- Multidimensional arrays

Lectures will emphasize the most important!

It is strongly recommended that you also read the recommending readings and complete the associated exercises.

JN

# Introduction

## Data structures

---

- **Data structures** are used to collect related data items for efficient access.
  - A data structure represents a collection of data values, e.g. a list, a table of data
  - The data structure may define the relationships among the data values, and often includes the operations or methods that can be applied to them
- *Recall:* An **algorithm** is a specified procedure to solve a problem
  - a sequence of definite and effective instructions or tasks, which terminates with the production of correct output from the given input.
- Many **computer programs** involve one or more **algorithms** processing large data structures.

***Algorithms + Data Structures = Programs 1976 book by Niklaus Wirth***

# Introduction

## Arrays

---

An **array** is a data structure

- that consists of related data items of the same **type**.
- Each data item is known as an **element** of the array.

A **Java Array** is an object.

- The array **type** can be primitive type e.g. **int** or reference type e.g. **String**.
- The array is of **fixed length**. Once created, its size cannot be changed.
- Java uses **[ ]** to distinguish an array.
- Each element has an **index** (position number) in the array.

### **class java.util.Arrays**

- contains various methods for manipulating arrays (such as sorting and searching)
- The Arrays methods are all **static** which means we can call them directly e.g. `Arrays.sort()`, `Arrays.toString()`

### **class ArrayList**

- An ArrayList is similar to an array but it allows **dynamic resizing**
- It automatically adjusts its size at execution time to store the elements.

# Array

## Example

- Consider an 8-element array named **arr** to hold integers
- **arr** is the name of the array
- **arr** has 8 elements
- **arr** type will be **int[ ]**
  - The element type is **int**
- **arr.length** gives the length (size) array **arr**, i.e. 8
- The elements are accessed using the array name and an index: **arrayName[index]**  
**arr[0], arr[1], arr[2], ... , arr[7]**
  - The index of the first element is 0
  - The index of the last element is **arr.length - 1**
- The value of **arr[1]** is 16.

int[ ]

arr

| Element | Value |
|---------|-------|
| arr[0]  | 11    |
| arr[1]  | 16    |
| arr[2]  | 3     |
| arr[3]  | 20    |
| arr[4]  | 4     |
| arr[5]  | 9     |
| arr[6]  | 8     |
| arr[7]  | 29    |

# Java Arrays

## Array names

- follow the same conventions as other variable names, **camelCase**
- start with lowercase letter, capitalize first letter of any subsequent word(s).
- Both singular and plural words are used for array names e.g. **score**, **scores**

## Array length

- An array object maintains its own length , stored in **length**
- **length** cannot be changed as it is a final instance variable.

```
System.out.println( arr.length );
```

## Array-access expression `arrayName[index]`

- is used to refer to a particular element in an array
- the **name** of the array followed by the **index** of the element in `[]`
- The index is also called a subscript.
- Can be an integer constant, variable or expression
- First element **index** is always **0**
- Last element **index** is always **length-1**
- Index is checked to ensure in the range **0** to **length-1**
- **JVM** prevents access to non-existent element.

```
arr[2] = 4;  
int entry = arr[1];
```

```
int x = 4;  
arr[ x + 2 ] += 2;  
// Adds 2 to arr[ 6 ]
```

# Arrays

## Activities

---

- Four major activities associated with arrays

1. Declaring arrays
2. Creating arrays
3. Initializing arrays
4. Manipulating array elements

Most often we:

*Declare and create together*

*Declare and initialize together*

- For readability, declare only one array per declaration statement.
  - Keep each declaration on a separate line.
  - Use a descriptive name.
  - Include a comment describing the variable being declared, if necessary.

# Arrays

## Declaring and Creating

- Arrays are **objects**, and you must declare and create them.
- **Declare** an array reference variable
  - Specify the type and variable name using **[]** (note empty, no size is specified).  
`int[] arr;`                      // Declares an array variable for an array of type int
  - In a declaration, the square brackets **[]** following a type indicate that the variable will refer to an array (i.e., store a reference to an array).
- **Create** an array using an **array creation expression**: **new int[8]**
  - An **array creation expression** specifies the **new** keyword, the element **type** and the number of elements i.e. the **length**
    - and returns a reference to the created object, which can be stored in an array variable.

`arr = new int[8];`                      // Normally assign to an array reference variable

- **Declare and create**

`int[] arr = new int[8];`    // Declare and create an array of 8 int elements

`String[] names = new String[ 100 ];` // an array of 100 String references



# Arrays

## Default values and Initializing arrays

### Default values

- When an array is created, each element of the array receives a default value
  - Zero for numeric primitive-type elements,
  - `false` for boolean elements,
  - `null` for references

### Initialize an array

- Array elements can be initialized to other values using an **initializer list**
- The initializer items are placed in a comma separated list enclosed in `{ }`
- The compiler counts the number of initializers in the list, then creates an array of that size, and assigns the initializers to the elements of the array.

### Declare and initialize examples

```
int[] arr = { 11, 16, 3, 20, 4, 9, 8, 29 };  
double[] heights = { 91.5, 60.4, 82.3 };  
String[] names = { "Hello", "Hola", "Ciao" };
```

You can explore these examples in the **SlidesExampleMix** class.

Step through the examples with a Debugger and display the arrays.

# Arrays

## Manipulating *the individual elements*

- You manipulate arrays by manipulating the individual elements
- To refer to an element use the array name, `[]` and the index

```
int[] arr = { 11, 16, 3, 20, 4, 9, 8, 29 };
```

```
arr[3]           // refers to the arr element at index 3 (the 4th element)
```

```
arr[7] = 5;      // assign the value 5 to the arr element at index 7 (last)
```

```
int elem = arr[2]; // assign to variable elem the arr element at index 2
```

- Trying to access a non-existent element results in a thrown exception
- The exception is `ArrayIndexOutOfBoundsException`

```
arr[10] = 4;      // attempt to access element at index 10 for above array
```

Results in the exception and a message explaining the reason:

**java.lang.ArrayIndexOutOfBoundsException: Index 10 out of bounds for length 8**

# Arrays

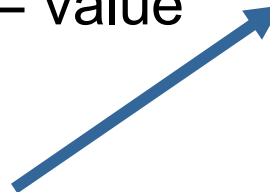
## Manipulating *the entire array* using **for**

- Consider the array

```
int[] arr = { 11, 16, 3, 20, 4, 9, 8, 29 };
```

- Print out the elements, one per line as `arr[index] = value`

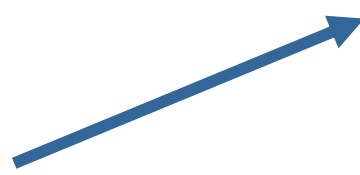
```
for (int i=0; i < arr.length; i++)  
    System.out.printf("arr[%d] = %d\n", i, arr[i]);
```



```
arr[0] = 11  
arr[1] = 16  
arr[2] = 3  
arr[3] = 20  
arr[4] = 4  
arr[5] = 9  
arr[6] = 8  
arr[7] = 5
```

- Set all element values to 1 (fill the array with 1s) and print

```
for (int i=0; i < arr.length; i++)  
    arr[i] = 1;  
  
for (int i=0; i < arr.length; i++)  
    System.out.printf("arr[%d] = %d\n", i, arr[i]);
```



```
arr[0] = 1  
arr[1] = 1  
arr[2] = 1  
arr[3] = 1  
arr[4] = 1  
arr[5] = 1  
arr[6] = 1  
arr[7] = 1
```

- Set all element values to twice their index. What are the element values?

```
for (int i=0; i < arr.length; i++)  
    arr[i] = 2*i;
```



```
?
```

# Arrays

## Summary of Activities

| Activity                                                                | Summary of Operations/Example                                                                                         |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Declare</b><br><i>an array variable</i>                              | <code>type [ ] arrayName;</code><br><code>int[ ] array1;</code>                                                       |
| <b>Create</b><br><i>an array object</i>                                 | <code>new type [size]    arrayName = new type [size];</code><br><code>new int[10]        array1 = new int[10];</code> |
| <b>Declare and create</b>                                               | <code>type [ ] arrayName = new type [size];</code><br><code>int[] array1 = new int[10];</code>                        |
| <b>Declare and Initialize</b><br><i>An array is implicitly created.</i> | <code>type [ ] arrayName = { Initialization List };</code><br><code>int[] array1 = { 2, 4, 8, 16*2+1 };</code>        |
| <b>Manipulate</b><br><i>Access/modify element values</i>                | <code>arrayName[index] = expression;</code><br><code>array1[0] = 15;    array1[3] = array1[0] * 3;</code>             |
| <b>Default initialisation</b><br><i>Array elements are initialised.</i> | <b>Arrays of primitives:</b> 0 numeric, false boolean<br><b>Arrays of references:</b> null                            |
| <b>Size</b>                                                             | <code>arrayName.length</code> <code>array1.length</code>                                                              |

# Array Examples

## Declare, Create, Initialize and Manipulate

1. Declare and create an array to hold four integers, call it **marks**

- Assign **6** to the first element, and **10** to the last.
- What are the values of the other two elements?

2. Declare and create an array to hold six double precision floating point numbers, call it **payments**.

- Assign **2.40** to the 3rd element and **1.25** to the 4th element.
- Assign to the element at index **5**, the element at index **2**.
- What are the values now stored in the array?

3. Declare and create an array to hold **3** String references, call it **colours**.

- Assign the values to the three elements in the order "**Red**", "**Green**" and "**Blue**"

4. Declare and initialise a similar String array with the same three colours.

### Declare & Create/Initialise, Manipulate

```
type [ ] arrayName = new type [size];  
int[] array1 = new int[10];
```

```
type [ ] arrayName = { Initialization List };  
int[] array1 = {2, 4, 8, 16};
```

```
arrayName[index] = expression;  
array1[0] = 15; array1[3] = array1[0] * 3;
```

# Array Example 1

## Declare, Create, Initialize and Manipulate

ArrayBasics.java

```
public static void main(String[] args)
{
    int[] marks;                // Declare an array variable
    marks = new int[4];         // Create the array with 4 elements
    marks[0] = 6;               // Assign 6 to the first element
    marks[3] = 10;              // Assign 10 to the last element
    // marks[1] and marks[2] will both be 0 //(from initialisation)

    double[] payments = new double[6]; // Declare/create payments
    payments[2] = 2.40;             // Assign 2.40 to the 3rd element
    payments[3] = 1.25;            // Assign 1.25 to the 4th element.
    payments[5] = payments[2];     // Assign to element at index 5, that at index 2
    // payments has the values 0.0, 0.0, 2.40, 1.25, 0.0, 2.40

    String[] colours = new String[3]; // Declare and create String array of length 3
    colours[0] = "Red";               // Assign reference to String "Red"
    colours[1] = "Green";
    colours[2] = "Blue";

    String[] colours2 = {"Red", "Green", "Blue"}; // Declare and initialise
}
```

# Array Exercise 1

## Declare, Create, Initialize and Manipulate

ArrayBasicsEx

ArrayBasicsEx.java

1. Declare and create the following arrays:
  - **arr1** to hold 10 integers.
  - **arr2** to hold 5 double precision floating point (real) numbers
  - **arr3** to hold 6 references to Strings
2. For the **arr1** array, assign 6 to the first element, and 12 to the last.
  - What are the values of the other elements?
3. Declare and initialise the following arrays:
  - **scores** to contain the integers 4, 6, 2, 1
  - **prices** to contain the numbers 1.21, 6.40, 8.12
  - **colours** to contain the strings "Yellow", "Brown", "White"
4. What is the code expression to determine how many elements are in the above arrays?.



# Array Exercise 1

## Declare, Create, Initialize and Manipulate (1 of 2)

```
public static void main(String[] args)
{
    //1. Declare and create the following arrays:
    //    arr1 to hold 10 integers
    //    arr2 to hold 5 double precision floating point (real) numbers
    //    arr3 to hold 6 references to strings
    int[] arr1 = new int[10];
    double[] arr2 = new double[5];
    String[] arr3 = new String[6];

    // Arrays.toString returns a string representation of the array
    System.out.println( "int []arr1 = new int[10]: " + Arrays.toString(arr1) );
    System.out.println( "double[] arr2 = new double[5]: " + Arrays.toString(arr2) );
    System.out.println( "String[] arr3 = new String[6]: " + Arrays.toString(arr3) );

    // 2. For the arr1 array, assign 6 to the first element, and 12 to the last.
    //    What are the values of the other elements?
    //    Primitive type: 0 if numeric e.g. int, double; false for boolean,
    //    Reference type: null e.g. String, Date etc
    arr1[0] = 6;
    arr1[9] = 12;
    System.out.println( "arr1 after assignments:  " + Arrays.toString(arr1) );
}
```



# Array Exercise 1

## Declare, Create, Initialize and Manipulate (2 of 2)

ArrayBasicsEx

ArrayBasicsEx.java

```
// 3. Declare and initialise the following arrays:
//     scores to contain the integers 4, 6, 2, 1
//     prices to contain the numbers 1.21, 6.40, 8.12
//     colours to contain the strings "Yellow", "Brown", "White"
int[] scores = { 4, 6, 2, 1 };
double[] prices = {1.21, 6.40, 8.12};
String[] colours = {"Yellow", "Brown", "White"};

//4. What is the code expression is used to determine
//    how many elements are in the above arrays?
//        scores.length,  prices.length,  colours.length

// Display the size of the array(the number of colours) and the colours
System.out.println( "scores length is " + scores.length );
System.out.println( "prices length is " + prices.length );
System.out.println( "colours length is " + colours.length );
```

You can explore the above code and more in the **ArrayBasicsEx** class.

Step through the code statements with a Debugger and display the arrays.

# Array Example 2

## Integer Array to hold marks

ArrayIntMarks

ArrayIntMarks.java

- Declare, create and change element values in an array

```
public static void main(String[] args)
{
    int[] marks = new int[4];    // Create an array to hold 4 elements (length = 4)

    System.out.println("After creating the marks array");
    for (int i=0; i < marks.length; i++)
        System.out.printf( "marks[%d] = %d\n", i, marks[i] );

    marks[0] = 6;                // Assign 6 to the first element, at index 0
    marks[1] = marks[0]/2 + 1;    // Assign expression result of 4 to element at index 1
    marks[2] = marks[1]*marks[1]; // Assign marks[1] squared to marks[2]
    ++marks[2];                 // Increment (add 1 to) the value of element at index 2
    marks[3] = 4;                // Assign 9 to the last element, at index 3
    marks[3] *= 2;               // Double element at index 3
    marks[3] += 3*3 - 2*2;        // Add expression result of 5 to marks[3]

    System.out.println("After assignment to the marks array elements");
    for (int i=0; i < marks.length; i++)
        System.out.printf( "marks[%d] = %2d\n", i, marks[i] );
}
```

After creating

|          |   |   |
|----------|---|---|
| marks[0] | = | 0 |
| marks[1] | = | 0 |
| marks[2] | = | 0 |
| marks[3] | = | 0 |

After assignment

|          |   |    |
|----------|---|----|
| marks[0] | = | 6  |
| marks[1] | = | 4  |
| marks[2] | = | 17 |
| marks[3] | = | 13 |

# Array Example 3

## Integer Array and Random Number Generation

ArrayIntRandom.java

- Generate 1M random numbers in the range 1 to 10 and check frequency
  - Must **import java.util.Random;**

```
public static void main(String[] args)
{
```

```
    Random rng = new Random();
```

```
    int[] scores = new int[1000000];
```

```
    for (int i=0; i < scores.length; i++)
```

```
        scores[i] = rng.nextInt(10) + 1;
```

```
    int[] frequency = new int[11];
```

```
    for (int i=0; i < scores.length; i++)
```

```
        ++frequency[ scores[i] ];
```

```
    System.out.printf("Frequency of %d scores in range [1,10]\n", scores.length);
```

```
    for (int i=1; i <= 10; i++)
```

```
        System.out.printf("frequency[%d] = %7d\n", i, frequency[i] );
```

```
}
```

```
Frequency of 1000000 scores in range [1,10]
frequency[ 1] = 100080
frequency[ 2] = 99899
frequency[ 3] = 100606
frequency[ 4] = 100117
frequency[ 5] = 100055
frequency[ 6] = 99739
frequency[ 7] = 99379
frequency[ 8] = 99724
frequency[ 9] = 100005
frequency[10] = 100396
```

```
// rng numbers in range [0,9] hence add 1
```

```
// Lets ignore element at index 0
```

```
// Increment frequency for each score
```



# Array Example 4

## Arrays of String References

ArrayString

ArrayString.java

```
public static void main(String[] args)
{
    // Declare and create an array to hold 3 String references, call it colours.
    // Assign the Colours to the three elements in order "Red", "Green" and "Blue"
    String[] colours = new String[3];
    colours[0] = "Red";
    colours[1] = "Green";
    colours[2] = "Blue";

    for (int i=0; i < colours.length; i++)
        System.out.printf("colours[%d]=\"%s\\n\", i, colours[i]);

    // Declare and initialise a similar String array with the three Subtractive colours
    String[] coloursSub = {"Cyan", "Magenta", "Yellow"};

    // Print in UPPER CASE (doesn't change the coloursSub elements)
    for (int i=0; i < coloursSub.length; i++)
        System.out.printf("coloursSub[%d]=\"%s\\n\", i, coloursSub[i].toUpperCase() );

    String[] colourPalette = new String[256];
    // Note all elements of colourPalette are null until assigned colours
    colourPalette[0] = "White";
}
```

```
colours[0]="Red"
colours[1]="Green"
colours[2]="Blue"
coloursSub[0]="CYAN"
coloursSub[1]="MAGENTA"
coloursSub[2]="YELLOW"
```

# Array Exercise 2

## Review Code and Draw Array

ArrayWeekEntryEx

ArrayWeekEntryEx.java

```
public static void main(String[] args)
{
    System.out.println( "\n<<<Exercise 3: weekEntry>>>" );

    final int NUMBER_ENTRIES = 4;
    double[] weekEntry = new double[NUMBER_ENTRIES];
    int week = 2;

    weekEntry[0] = 17.5;
    weekEntry[1] = weekEntry[0] * 4.0;
    weekEntry[week] = weekEntry[0] / weekEntry[1];
    weekEntry[++week] = 16.0;
    weekEntry[0] *= 2.0;

    display("weekEntry after assignment ", weekEntry);
    clear(weekEntry);
    display("weekEntry after calling clear()", weekEntry);
}
```

What are the element values of the array referenced by **weekEntry** at this line?

Draw the array using a table.

| name | type    |       |
|------|---------|-------|
|      | Element | Value |
|      |         |       |
|      |         |       |
|      |         |       |
|      |         |       |

# Array Exercise 2

## display and clear methods

ArrayWeekEntryEx

ArrayWeekEntryEx.java

```
/**
 * Display a name and the element values of a double array
 * @param name      a name/description of the array
 * @param array      the array to display
 */
public static void display(String name, double[] array)
{
    System.out.printf("Values of array %s are:\n", name);
    for(int i=0; i < array.length; i++)
        System.out.printf("%-4d %5.2f\n", i+1, array[i]);
}
```

```
/**
 * Clear the array by setting all element values to 0.0
 * @param array      the array to initialise
 */
public static void clear(double[] array)
{
    for(int i=0; i < array.length; i++)
        array[i] = 0.0;
}
```



# Array Exercise 2

## Review Code and Draw Array: Solution

ArrayWeekEntryEx

ArrayWeekEntryEx.java

```
public static void main(String[] args)
{
    System.out.println( "\n<<<Exercise 3: weekEntry>>>" );

    final int NUMBER_ENTRIES = 4;
    double[] weekEntry = new double[NUMBER_ENTRIES];
    int week = 2;

    weekEntry[0] = 17.5;
    weekEntry[1] = weekEntry[0] * 4.0;
    weekEntry[week] = weekEntry[0] / weekEntry[1];
    weekEntry[++week] = 16.0;
    weekEntry[0] *= 2.0;

    display("weekEntry after assignment ", weekEntry);
    clear(weekEntry);
    display("weekEntry after calling clear()", weekEntry);
}
```

weekEntry

double[]

| Element      | Value |
|--------------|-------|
| weekEntry[0] | 35.00 |
| weekEntry[1] | 70.00 |
| weekEntry[2] | 0.25  |
| weekEntry[3] | 16.00 |

```
public static void display(String name, double[] array)
{
    System.out.printf("Values of array %s are:\n", name);
    for(int i=0; i < array.length; i++)
        System.out.printf("%-4d %5.2f\n", i+1, array[i]);
}
```

```
<<<Exercise 3: weekEntry>>>
Values of array weekEntry after assignment are:
1 35.00
2 70.00
3 0.25
4 16.00
Values of array weekEntry after calling clear() are:
1 0.00
2 0.00
3 0.00
4 0.00
```

# Array Exercise 3

## Copy an array

---

Write a code fragment for the following:

1. Create two arrays **arrA** and **arrB** as follows:
  - **arrA** is of type `int` and must be initialised to hold the integer values: 6 , 3, -1, 10
  - **arrB** is large enough to hold the elements of **arrA**.
2. Copy the elements of **arrA** to **arrB**
3. Display the values in **arrA** (and **arrB**)
4. Draw a pictorial representation of either array

What code would you need to add to execute the above?



# Array Exercise 3

## Copy an array: Solution

ArrayCopyEx

ArrayCopyEx.java

```
public static void main(String[] args)
{
```

```
    System.out.println( "\n<<<Copy array example>>>" );
```

```
    int[] arrA = { 6, 3, -1, 10 };           // Declare and initialise
    int[] arrB = new int[ arrA.length ];     // Declare and create
```

```
    //arrB = arrA;    // Allowed but not what you expect.
```

```
    for( int i = 0; i < arrB.length; ++i)    // Must copy element by element.
        arrB[i] = arrA[i];
```

```
    for( int i = 0; i < arrA.length; ++i)
        System.out.printf("Value of arrA[%d] is %d\n", i, arrA[i]);
```

```
    for( int i = 0; i < arrB.length; ++i)
        System.out.printf("Value of arrB[%d] is %d\n", i, arrB[i]);
```

```
}
```

```
<<<Copy array example>>>
Value of arrA[0] is 6
Value of arrA[1] is 3
Value of arrA[2] is -1
Value of arrA[3] is 10
Value of arrB[0] is 6
Value of arrB[1] is 3
Value of arrB[2] is -1
Value of arrB[3] is 10
```

arrA

int[]

| Element | Value |
|---------|-------|
| arrA[0] | 6     |
| arrA[1] | 3     |
| arrA[2] | -1    |
| arrA[3] | 10    |

# Array Example 5

## Declare, Create, Initialize and Manipulate

ArrayManipulate

ArrayManipulate.java

- Demonstrates double arrays, and nested control statements.
  1. Input and store a group of real numbers within a specified range.
  2. Find the min, max and average.
  3. Remove duplicate values and display the average with no duplicates.
- Based on a previous student challenge.

```
/**
 * Explore Java arrays manipulations.
 *
 * Code, execute and debug the application below using a one-dimensional array.
 *
 * 1. Prompt the user for the number of floating point numbers to enter
 *    each between 0.1 and 100.0, inclusive.
 * 2. Input and store in the array the user defined number of entries in range.
 * 3. Display in table form, the index and the value of each element in the array.
 * 4. Calculate and display the minimum, maximum and average of the array elements.
 * 5. Count and eliminate any duplicates in the array.
 * 6. Calculate and display the average after eliminating the duplicates.
 */
```

# Array Example 5

ArrayManipulate

## Declare, Create, Initialize and Manipulate (1 of 3) ArrayManipulate.java

```
import java.util.Scanner; // Going to use the Scanner object, must import
public class ArrayManipulate
{
    public static void main( String[] args )
    {
        System.out.println( "Process an array of real numbers\n in the range [0.1,100.0]" );

        // Request how many numbers to enter
        System.out.print("How many numbers: ");
        Scanner input = new Scanner(System.in);
        int numNumbers = input.nextInt();

        // Declare and create the array for requested size
        double[] numbers = new double[numNumbers];

        // Request numbers in range 0.1 to 100.0
        for (int i=0; i < numbers.length; i++)
        {
            do
            {
                System.out.printf( "Enter number %d: ", i + 1 );
                numbers[i] = input.nextDouble();
            }
            while( numbers[i] < 0.1 || numbers[i] > 100.0);
        }
    }
}
```

Input and store a user defined number of floating point values in an array, each between 0.1 and 100.0, inclusive

```
Process an array of real numbers
in the range [0.1,100.0]
How many numbers: 6
Enter number 1: 1.1
Enter number 2: 2.2
Enter number 3: 3.3
Enter number 4: 4.4
Enter number 5: 3.3
Enter number 6: 1.1
```



# Array Example 5

## Declare, Create, Initialize and Manipulate (2 of 3) ArrayManipulate.java

```
// Calculate the min, max and average
double min = Double.MAX_VALUE; // Initialise to largest double
double max = Double.MIN_VALUE; // Initialise to smallest double
double sum = 0.0;
```

```
for (int i=0; i < numbers.length; i++)
{
    sum += numbers[i];
    if (numbers[i] < min)
        min = numbers[i];
    if (numbers[i] > max)
        max = numbers[i];
}
```

The array **double[] numbers**; has been created earlier.

Now calculate the min, max and average of the array elements.

Display the array elements, and the min, max and average.

```
// Display the array elements and the min, max and average
System.out.printf("%5s %10s\n", "Index", "Element");
for (int i=0; i < numbers.length; i++)
{
    System.out.printf("%5d %10.2f\n", i, numbers[i] );
}
System.out.printf("The minimum is %.2f\n", min);
System.out.printf("The maximum is %.2f\n", max);
System.out.printf("The average is %.2f\n", sum/numbers.length);
```

| Index               | Element |
|---------------------|---------|
| 0                   | 1.10    |
| 1                   | 2.20    |
| 2                   | 3.30    |
| 3                   | 4.40    |
| 4                   | 3.30    |
| 5                   | 1.10    |
| The minimum is 1.10 |         |
| The maximum is 4.40 |         |
| The average is 2.57 |         |

# Array Example 5

ArrayManipulate

## Declare, Create, Initialize and Manipulate (3 of 3)

ArrayManipulate.java

```
// Remove any duplicates from the array and sum
int duplicatesRemoved = 0;
for (int i=0; i < numbers.length - 1; i++)
{
    for (int j = i + 1; j < numbers.length; j++)
    {
        if (numbers[j] != 0 && numbers[j] == numbers[i])
        {
            sum -= numbers[j];
            numbers[j] = 0.0;
            ++duplicatesRemoved;
        }
    }
}
System.out.printf("%d Duplicates Removed\n", duplicatesRemoved);
System.out.printf("The average (no duplicates) is %.2f\n",
    sum/(numbers.length - duplicatesRemoved) );
}
```

This is a crude  
duplicate removal!

2 Duplicates Removed  
The average (no duplicates) is 2.75

# for-each enhanced for Statement

## Pseudocode

*for each element in the array  
process the element*

for ( **type element : array** )  
**statement**

The **for** iteratively walks through the given **array** returning each element value in turn in the variable **element** and then executes the **for** body **statement**.

## Java

```
{  
    int[] arr = { 11, 16, 3, 20, 4, 9, 8, 29 };  
    int total = 0;  
  
    for ( int number : arr )  
        total += number;  
  
    // is equivalent to  
    for ( int counter = 0; counter < arr.length; counter++ )  
        total += arr[ counter ];  
}
```

*Create variable and initialize **total = 0***

*for each number element in **arr**  
add number element to total*



# Passing Arrays to Methods

ArrayPassingEx

ArrayPassingEx.java

- Arrays can be passed to methods and returned from methods
- To pass an array argument to a method
  - Specify the array name without brackets
  - Since an array object maintains its own length, we do not have to specify it.

## Example

1. Given an array `dailyTempC` holding daily temperatures in °C over 7 days:

```
double[] dailyTempC = {14.5, 13.0, 16.5, 18.5, 22.0, 15.5, 12.0 };
```

2. `calcAverage( dailyTempC )` will call the `calcAverage` method to find the average temperature with the array `dailyTempC` passed as an argument:

```
System.out.printf("Average temp °C is: %.1f\n", calcAverage( dailyTempC ) );
```

3. 

```
public static double calcAverage( double[] array)
{
    double total = 0;
    for (double element : array)
        total += element;
    return total/array.length;
}
```

Average temp °C is: 16.0

The `calcAverage` method specifies an array parameter called `array` of type `double`.

# Array Exercise 4

## Passing Arrays to Methods

ArrayPassingEx

ArrayPassingEx.java

- Consider a method to find the sum total of the array elements.

```
/**
 * Find the sum total of the elements of an int array
 * @param arr    the array of elements
 * @return       the sum total
 */
public static int findSum( int[] arrParameter)
{
    int total = 0;
    for (int i = 0; i < arrParameter.length; i++)
        total += arrParameter[i];
    return total;
}
```

- We call the method e.g. **findSum( testArray )** and the output is:

```
int []testArray = new int[8];

System.out.print( "\nThe original testArray is: \n " + Arrays.toString( testArray ) );
System.out.println( ", Total is: " + findSum( testArray ) );
```

```
The original testArray is:
[0, 0, 0, 0, 0, 0, 0, 0], Total is: 0
```

# Array Exercise 4

## Passing Arrays to Methods

ArrayPassingEx

ArrayPassingEx.java

- Complete the method body below to modify each element, of an integer array, passed as a parameter, to the value of its index + 1.

```
/**
 * Modify each element of the integer array to the value of its index + 1
 * i.e. array[0] = 1, array[1] = 2, array[2] = 3, ...
 * @param array    the array of elements to be initialised
 */
public static void modifyArray1( int[] array )
```

- Write a method to modify the above method to modify only every third element to the value of its index + 1, and set the others to -1.

```
/**
 * Modify every third element of the integer array to the
 * value of its index + 1 and set the others to -1.
 * @param array    the array of elements to be modified
 */
```

# Array Exercise 4

## Passing Arrays to Methods (Solution)

ArrayPassingEx

ArrayPassingEx.java

```
/**
 * Modify each element of the integer array to the value of its index + 1
 * i.e. array[0] = 1, array[1] = 2, array[2] = 3, ...
 * @param array the array of elements to be initialised
 */
public static void modifyArray1( int[] array )
{
    for (int index=0; index < array.length; index++)
    {
        array[index] = index + 1;
    }
}
```

```
/**
 * Modify every third element of the integer array to the
 * value of its index + 1 and set the others to -1.
 * @param array the array of elements to be modified
 */
public static void modifyArray2( int[] array )
{
    for (int i=0; i < array.length; i++)
    {
        if ( i%3 == 2) // if remainder of i/3 is 2
            array[i] = i + 1;
        else
            array[i] = -1;
    }
}
```

<<<Exercise 4: passing arrays to methods >>>

The original testArray is:

[0, 0, 0, 0, 0, 0, 0, 0], Total is: 0

After call to modifyArray1(testArray):

Array is [1, 2, 3, 4, 5, 6, 7, 8], Total is: 36

After call to modifyArray2(testArray):

Array is [-1, -1, 3, -1, -1, 6, -1, -1], Total is: 3



# Returning Arrays from Methods

- An array can be returned from a method.
  - More correctly an array reference is returned from the method.
- This can refer to an existing array or one just created by the method.
- To return an array we declare the method as returning an array and use the return statement to return the array reference.

## Example method: `int[] createIntArray()`

- Return an integer array with element values: 4, 2, 5, 9, 16, 21.

```
/**
 * Demonstrate returning an integer array from a method.
 * Declare and initialise an array of integers with element
 * values: 4, 2, 5, 9, 16, 21 and return the array
 * @return the created array
 */
public static int[] createIntArray( )
{
    int[] numbers = { 4, 2, 5, 9, 16, 21 };
    return numbers;
}
```



# Array Exercise 5

## Returning Arrays from Methods

- Similar to:

```
public static int[] createIntArray( )  
{  
    int[] numbers = { 4, 2, 5, 9, 16, 21 };  
    return numbers;  
}
```

- Write a method **double[ ] createDoubleArray( )**

to declare and initialise an array of double with 4 element values:  
4.1, 2, 5.4, 9.2, 4E6 and return the array.

- Write a method **int[ ] createRandomArray( int numElements)**

to declare and create an array of integers of **numElements** in length  
and fill the array with random values between 1 and 100.

Hint assume: **import java.util.Random;** and use method **nextInt(int bound)**

**int nextInt(int bound)** Returns a pseudorandom, uniformly distributed int value  
between 0 (inclusive) and the specified value (exclusive),  
drawn from this random number generator's sequence.

# Array Exercise 5

## Returning Arrays from Methods (Solution)

ArrayReturningEx.java

```
/**
 * Demonstrate returning a double array from a method.
 * Declares and initialises an array with 5 real elements with
 * values: 4.1, 2, 5.4, 9.2, 4E6 and return the array.
 * @return the created array
 */
public static double[] createDoubleArray( )
{
    double[] myRealValueArray = { 4.1, 2, 5.4, 9.2, 4E6 };
    return myRealValueArray;
}

/**
 * Demonstrate returning a random array of elements from 1 to 100
 * Create an array of requested size and fill with random numbers
 * @param numElements the length of the array to create
 * @return the created array
 */
public static int[] createRandomArray(int numElements)
{
    int[] array = new int[numElements];
    Random rng = new Random();
    for (int i=0; i< numElements; i++)
        array[i] = rng.nextInt(100) + 1; // numbers in range 1 to 100
    return array;
}
```

See the class  
**ArrayReturningEx**  
for more details,  
including code to call  
the methods and  
display the arrays.

# Array Exercise 5

## Returning Arrays from Methods (Solution)

ArrayReturningEx

ArrayReturningEx.java

```
/**
 * Demonstrate returning a double array from a method.
 * Declares and initialises an array with 5 real elements with
 * values: 4.1, 2, 5.4, 9.2, 4E6 and return the array.
 * @return the created array
 */
public static double[] createDoubleArray( )
{
    double[] myRealValueArray = { 4.1, 2, 5.4, 9.2, 4E6 };
    return myRealValueArray;
}
```

```
/**
 * Demonstrate returning a random array of elements from 1 to 100
 * Create an array of requested size and fill with random numbers
 * @param numElements the length of the array to create
 * @return the created array
 */
public static int[] createRandomArray(int numElements)
{
    int[] array = new int[numElements];
    Random rng = new Random();
    for (int i=0; i< numElements; i++)
        array[i] = rng.nextInt(100) + 1; // numbers in range 1 to 100
    return array;
}
```

See the class  
**ArrayReturningEx**  
for more details,  
including code to call  
the methods and  
display the arrays.

# Passing Arguments to Methods

## Two methods (and generally is confusing!)

---

- **Pass-by-value** (aka call-by-value)
  - A copy of the argument's value is passed to the called method.
  - Hence, the method has a copy and cannot modify the value in the caller.
- **Pass-by-reference** (aka call-by-reference)
  - The called method gets a reference to the argument's location and can modify the value, if required.
- All arguments in Java are effectively passed by value.
- A method call can pass two types of values to a method
  - Copies of primitive values and copies of references to objects
  - Objects cannot be copied and passed to a method, only the reference is copied.
- For references, when the called method begins execution, the parameter in the called method and the argument in the calling method refer to the same object in memory.
- If a method modifies the reference-type parameter so that it refers to another object, only the parameter refers to the new object
  - The reference stored in the caller's variable still refers to the original object.
- Although an object's reference is passed by value, a method can still interact with the referenced object by calling its public methods.

# Passing Arguments to Methods

## Examples

- **Pass-by-value** primitives

- Consider a method `int cube(int x)` to calculate the cube of the parameter `x`.
- Example calls to the method, in each case `x` will be a copy of the argument value.  
`cube(4)` // `x` will be 4  
`cube(2*2+1)` // `x` will be 5, the result of the expression  
`int value = 10;`  
`cube(value)` // `x` will be 10, modifying `x` will not change `value`.

- **Pass-by-value** references

- Consider a method `int calcAverage( int[] array )`  
which calculates the average of the elements in the parameter `array`.
- Each call to the method `calcAverage` must be a reference variable of type `int[]`  
`int[] numbers = { 4, 2, 5, 9, 16, 21 };`  
`calcAverage(numbers);` // `array` and `numbers` will refer to the same array
- The value of reference variable `array` will the value of reference variable `numbers`



# class Arrays

## java.util.Arrays

- The **java.util.Arrays** class contains various methods for manipulating arrays
- All are `static` methods: they be called using class name **Arrays**.

**Arrays.sort( arrayNameToSort );**

- Methods exist (overloaded) for primitive-type arrays and for arrays of objects.  
**byte, char, double, float, int, long, short, and Object**

| returns                 | name                             | description                                                                                                   |
|-------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------|
| String                  | <b>toString</b> (array)          | Returns a string representation of the contents of the specified array, delimited by [] and separated by ", " |
| void                    | <b>sort</b> (array)              | sort an array in ascending order                                                                              |
| int                     | <b>binarySearch</b> (array, key) | For efficient searching of an already sorted array                                                            |
| boolean                 | <b>equals</b> (array1, array2)   | A Boolean method to compare two arrays                                                                        |
| <i>Ref to new array</i> | <b>copyOf</b> (array, newLength) | Copies the specified array, truncating or padding (if necessary) so the copy has the specified length.        |
| void                    | <b>fill</b> (array, value)       | Assigns the value to each element of the array.                                                               |

- In addition, the **System** class has a useful **arraycopy** method to flexibly copy arrays

# class Arrays

java.util.Arrays

ArraysMethods

ArraysMethod.java

Demonstrate class Arrays methods using an array of type **double**

- Declare and initialise an array to hold 7 double numbers representing daily temperatures, read daily over the past week.

```
import java.util.Arrays;
public class ArraysMethods
{
    public static void main(String[] args)
    {
        double[] dailyTempC = {14.5, 13.0, 16.5, 18.5, 22.0, 15.5, 12.0 };
    }
}
```

- Explore various class Arrays methods:  
toString, sort, binarySearch, fill, equals, copyOf

# class Arrays

## java.util.Arrays

ArraysMethods

ArraysMethod.java

- Generate String String Arrays.**toString**(double[] a)
  - Generate a string representation of an array
  - Versions of **toString** exist for all arrays of primitive type and **Object[]**.

```
// Print an array using Arrays.toString (returns a String representation)
System.out.printf("Daily temperatures over %d days\n", dailyTempC.length);
System.out.println( Arrays.toString(dailyTempC) );
```

```
// Sort an array in ascending order using Arrays.sort
System.out.println("Sorting the temperatures in ascending order");
Arrays.sort(dailyTempC);
System.out.println( Arrays.toString(dailyTempC) );
```

```
Daily temperatures over 7 days
[14.5, 13.0, 16.5, 18.5, 22.0, 15.5, 12.0]
```

# class Arrays

## java.util.Arrays

ArraysMethods

ArraysMethod.java

- **Sort** an Array      void Arrays.**sort**(double[] a)
- **Search** an Array    int Arrays.**binarySearch**(double[] a, double key)

```
// Sort an array in ascending order using Arrays.sort
System.out.println("Sorting the temperatures in ascending order");
Arrays.sort(dailyTempC);
System.out.println( Arrays.toString(dailyTempC) );

// Search a sorted array for a value using Arrays.binarySearch method
int matchIndex = Arrays.binarySearch(dailyTempC, 18.5);
if (matchIndex == -1)
    System.out.printf("18.5 was not found");
else
    System.out.printf("18.5 was found at dailyTempC[%d]\n", matchIndex);
```

```
Sorting the temperatures in ascending order
[12.0, 13.0, 14.5, 15.5, 16.5, 18.5, 22.0]
18.5 was found at dailyTempC[5]
```

# class Arrays

## java.util.Arrays

ArraysMethods

ArraysMethod.java

- **Fill** an Array      void Arrays.**fill**(double[] a, double val)
- **Compare** Arrays    boolean Arrays.**equals**(double[] a, double[] a2)
- **Duplicate** Array    double[ ] Arrays.**copyOf**(double[] original, int newLength)

```
// Compare arrays using the Arrays.equals method
double[] expectedTempC = new double[7]; // Create a second array to compare
Arrays.fill(expectedTempC, 20.0);        // Assign all expected to 20C
System.out.printf("\nDaily temperatures were all 20C is %b\n",
    Arrays.equals(dailyTempC, expectedTempC) );

// Duplicate the dailyTempC array
double[] dailyTempDuplicate = Arrays.copyOf(dailyTempC, dailyTempC.length);
System.out.printf("Duplicate temperatures array created correctly is %b\n",
    Arrays.equals(dailyTempC, dailyTempDuplicate) );
```

```
Daily temperatures were all 20C is false
Duplicate temperatures array created correctly is true
```



# ArrayList

## java.util.ArrayList

- An **ArrayList** dynamically grows and shrinks to accommodate the addition and the removal of elements, hence overcoming the fixed length of arrays.
- The **java.util.ArrayList** class allows the creation of arrays that change their size during execution.
- Use **ArrayList<E>** where **E** is the element type to store in the array.
  - The element type **E** must be a reference type.
  - There are equivalent reference types for all the primitives i.e.  
Boolean, Byte, Character, Double, Float, **Integer**, Long, Short
  - Its **capacity** is how many elements it can currently hold
  - Its **size** is many elements it currently has.
- As we can specify the element type **E**, the class is known as a **generic class**.
- Methods are used for all manipulation e.g. to add and remove elements.
- Create array lists to store **Integer**, **Double** and **String**. Initial capacity will be 10.

```
ArrayList<Integer> aMarks = new ArrayList<Integer>();  
ArrayList<Double> aPayments = new ArrayList<Double>();  
ArrayList<String> aColours = new ArrayList<String>();
```

# ArrayList

## Methods

|                                                                                                                             |                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>void add(int index, E element)</b>                                                                                       | Inserts the specified element at the specified position in this list.                                    |
| <b>boolean add(E e)</b>                                                                                                     | Appends the specified element to the end of this list.                                                   |
| <b>E get(int index)</b>                                                                                                     | Returns the element at the specified index position in this list.                                        |
| <b>E set(int index, E element)</b>                                                                                          | Replaces the element at the specified position with the specified element.                               |
| <b>E remove(int index)</b>                                                                                                  | Removes the element at the specified position in this list.                                              |
| <b>boolean remove(Object o)</b>                                                                                             | Removes the first occurrence of the specified element from this list, if it is present.                  |
| <b>int indexOf(Object o)</b>                                                                                                | Returns the index of the first occurrence of the specified element in this list, or -1 if not contained. |
| <b>int size()</b>                                                                                                           | Returns the number of elements in this list.                                                             |
| Other useful methods: <b>clear()</b> , <b>contains()</b> , <b>ensureCapacity()</b> , <b>isEmpty()</b> , <b>trimToSize()</b> |                                                                                                          |

# ArrayList Examples

ArrayLists

## ArrayBasics int[] example using ArrayList

ArrayLists.java

```
int[] marks = new int[4];
```

```
ArrayList<Integer> aMarks = new ArrayList<Integer>();
```

```
public static void main(String[] args)
{
    // ArrayBasics example marks array below using an ArrayList
    // int[] marks;
    // marks = new int[4];
    // marks[0] = 6;
    // marks[3] = 10;
    ArrayList<Integer> aMarks = new ArrayList<Integer>(); //initial capacity of 10
    for(int i=0; i < 4; i++) // Insert 6 elements so size is 6
    {
        aMarks.add(0);

        aMarks.set(0, 6); // Replace the element 6 at index 0
        aMarks.set(3, 10); // Replace the element at index 3 with 10
    }
    System.out.println( aMarks ); // [6, 0, 0, 10]
```

[6, 0, 0, 10]

# ArrayList Examples

ArrayLists

ArrayBasics **double[]** example using **ArrayList**

ArrayLists.java

```
double[] payments = new double[6];
```

```
ArrayList<Double> aPayments = new ArrayList<Double>();
```

```
// ArrayBasics example payments array below using an ArrayList
```

```
// double[] payments = new double[6];
```

```
// payments[2] = 2.40;
```

```
// payments[3] = 1.25;
```

```
// payments[5] = payments[2];
```

```
// payments has the values 0.0, 0.0, 2.40, 1.25, 0.0, 2.40
```

```
ArrayList<Double> aPayments = new ArrayList<Double>();
```

```
for(int i=0; i < 6; i++)           // Insert 6 elements so size is 6
```

```
    aPayments.add(0.0);
```

```
aPayments.set(2, 2.40);           // Replace the element at index 2 with 2.40
```

```
aPayments.set(3, 1.25);           // Replace the element at index 3 with 1.25
```

```
aPayments.set(5, aPayments.get(2)); // Replace index 5 element with index 2 element
```

```
System.out.printf( "%s (size:%d)\n", aPayments, aPayments.size() );
```

```
[0.0, 0.0, 2.4, 1.25, 0.0, 2.4] (size:6)
```

# ArrayList Example

## ArrayBasics **String[]** example using **ArrayList**

ArrayLists

ArrayLists.java

```
String[] colours = new String[3];
```

```
ArrayList<String> aColours = new ArrayList<String>();
```

```
// ArrayBasics example colours array below using an ArrayList
// String[] colours = new String[3];
// colours[0] = "Red";
// colours[1] = "Green";
// colours[2] = "Blue";
ArrayList<String> aColours = new ArrayList<String>();
aColours.add("Red");           // Since empty, add "Red" at index 0
aColours.add("Green");        // Add "Green" at index 1, the 2nd position
aColours.add(2, "Blue");      // Add "Blue" at index 2, the 3rd position
System.out.println( aColours ); // [Red, Green, Blue]
```

**[Red, Green, Blue]**



# ArrayList Examples

## Add and Remove

ArrayLists

ArrayLists.java

- Demonstrate add and removal in the **aColours** list

```
aColours.add(1, "Yellow");           // Insert "Yellow" at index 1
aColours.add(3, "Purple");           // Insert "Purple" at index 3
System.out.println( aColours );      // [Red, Yellow, Green, Purple, Blue]
aColours.remove("Green");            // Remove first occurrence of "Green" in list
aColours.remove(3);                  // Remove element at index 3
System.out.println( aColours );      // [Red, Yellow, Purple]
System.out.printf("Index of \"Purple\" is %d\n", aColours.indexOf("Purple"));
System.out.printf("Index of \"Green\" is %d\n", aColours.indexOf("Green"));
}
```

```
[Red, Yellow, Green, Purple, Blue]
[Red, Yellow, Purple]
Index of "Purple" is 2
Index of "Green" is -1
```

# Multi-dimensional arrays

---

- The arrays to date have a single dimension e.g. `int[] arr = new int[8];`
- Many problems require multidimensional arrays
- **Multidimensional arrays** can have more than two dimensions.
- Java supports multidimensional arrays, as arrays of arrays, always initialised.
  - Specify one-dimensional arrays whose elements are also one-dimensional arrays.  
where recursively each of the elements can be a one-dimensional array.
- **`java.lang.ArrayIndexOutOfBoundsException`** thrown if any index is invalid
  - tries to access an element with an index that isn't valid for that dimension.
- **Two-dimensional arrays** are used to represent two-dimensional tables of values where information is arranged in rows and columns.
  - Each table element is identified with two indices for row and column
  - By convention, the order is row first and column second.
  - Data is stored in in row major order.
  - An array with **m** rows and **n** columns is called an **m-by-n** array.

# Two dimensional (2-di) arrays

Regular: Declare, Create and Manipulate

ArrayMultiDi

ArrayMultiDi.java

- Consider a regular two dimensional **3-by-4** array called table:  
each row has the same number of columns e.g. 3 rows and 4 columns

Declare and create the array:

```
int[ ][ ] table = new int[3][4];
```

| <b>table</b> | Column 0    | Column 1    | Column 2    | Column 3    |
|--------------|-------------|-------------|-------------|-------------|
| Row 0        | table[0][0] | table[0][1] | table[0][2] | table[0][3] |
| Row 1        | table[1][0] | table[1][1] | table[1][2] | table[1][3] |
| Row 2        | table[2][0] | table[2][1] | table[2][2] | table[1][3] |

Manipulate (elements):

```
table[0][1] = 5;
```

```
table[0][3]++;
```

```
table[1][2] = 6;
```

```
table[1][2] -= 2;
```

```
table[2][1] = table[0][1]*2;
```

| <b>table</b> | Column 0 | Column 1 | Column 2 | Column 3 |
|--------------|----------|----------|----------|----------|
| Row 0        | 0        | 5        | 0        | 1        |
| Row 1        | 0        | 0        | 4        | 0        |
| Row 2        | 0        | 10       | 0        | 0        |

# Two dimensional arrays

## Irregular: Declare, Create and Manipulate

- Consider a two dimensional table where the column sizes vary per row:  
row 0 has 4 elements, row 1 has 3, row 2 has 2

Declare and create the 2-di array (create each row as a one-dimensional array):

```
int[][] table = new int[3][];
```

```
table[0] = new int[4];
```

```
table[1] = new int[3];
```

```
table[2] = new int[2];
```

Manipulate (elements):

```
table[0][1] = 5;
```

```
table[0][3]++;
```

```
table[1][2] = 6;
```

```
table[1][2] -= 2;
```

```
table[2][1] = table[0][1]*2;
```

| <b>table</b>             | Column 0    | Column 1    | Column 2    | Column 3    |
|--------------------------|-------------|-------------|-------------|-------------|
| <b>Row 0</b><br>table[0] | table[0][0] | table[0][1] | table[0][2] | table[0][3] |
| <b>Row 1</b><br>table[1] | table[1][0] | table[1][1] | table[1][2] |             |
| <b>Row 2</b><br>table[2] | table[2][0] | table[2][1] |             |             |

| <b>table</b> | Column 0 | Column 1 | Column 2 | Column 3 |
|--------------|----------|----------|----------|----------|
| <b>Row 0</b> | 0        | 5        | 0        | 1        |
| <b>Row 1</b> | 0        | 0        | 4        |          |
| <b>Row 2</b> | 0        | 10       |          |          |

# Two dimensional arrays

## Declare and initialise

ArrayMultiDi.java

- Use nested array initialisers delimited by { }, each row is delimited by { }
- Regular: declare and initialise :

```
int[][] table = { {-1, 5, 0, 1}, {-2, 0, 4, 0}, {-3, 10, 0, 0} };
```

| <b>table</b> | Column 0 | Column 1 | Column 2 | Column 3 |
|--------------|----------|----------|----------|----------|
| Row 0        | -1       | 5        | 0        | 1        |
| Row 1        | -2       | 0        | 4        | 0        |
| Row 2        | -3       | 10       | 0        | -1       |

- Irregular: declare and initialise

```
int[][] table = { {-1, 5, 0, 1}, {-2, 0, 4}, {-3, 10} };
```

| <b>table</b> | Column 0 | Column 1 | Column 2 | Column 3 |
|--------------|----------|----------|----------|----------|
| Row 0        | -1       | 5        | 0        | 1        |
| Row 1        | -2       | 0        | 4        |          |
| Row 2        | -3       | 10       |          |          |



# Three dimensional (3-di) arrays

## Regular: Declare, Create and Manipulate

ArrayMultiDi.java

```
final int N_STUDENTS = 5;  
final int N_MODULES = 2;  
final int N_WEEKS = 8;
```

Consider a regular three dimensional array to maintain weekly attendance for 5 students taking 2 modules over 8 weeks. Use boolean true or false to indicate attendance.

```
boolean[][][] attend = new boolean[N_STUDENTS][N_MODULES][N_WEEKS];
```

```
// Assume all weeks attended! Cannot use <enhanced for> to assign to array element  
for (int i = 0; i < attend.length; i++)  
    for (int j = 0; j < attend[i].length; j++)  
        for (int k = 0; k < attend[i][j].length; k++)  
            attend[i][j][k] = true;
```

```
// Use enhanced for to count how many weeks not attended in total  
int missedWeeks = 0;  
for (boolean[][] student : attend )  
    for (boolean[] module : student)  
        for (boolean week : module )  
            if (!week)  
                ++missedWeeks;
```

Note: the **for** or enhanced **for** control statements are commonly used to iterate through arrays.

```
System.out.printf("%d missed of %d\n", missedWeeks, N_STUDENTS*N_MODULES*N_WEEKS);
```

# Three dimensional (3-di) arrays

Regular: Declare and initialise

ArrayMultiDi

ArrayMultiDi.java

Store sensor readings from 3 machines each with 4 sensors with two readings/day

|           | Sensor 1 | Sensor 2 | Sensor 3 | Sensor 4 |
|-----------|----------|----------|----------|----------|
| Machine 1 | 21.5     | 20.5     | 24.4     | 22.7     |
|           | 24.5     | 23.1     | 24.2     | 20.6     |
| Machine 2 | 31.2     | 32.5     | 36.8     | 30.5     |
|           | 34.5     | 34.1     | 38.4     | 31.6     |
| Machine 3 | 0.0      | 0.0      | 0.0      | 24.0     |
|           | 0.0      | 0.0      | 0.0      | 29.9     |

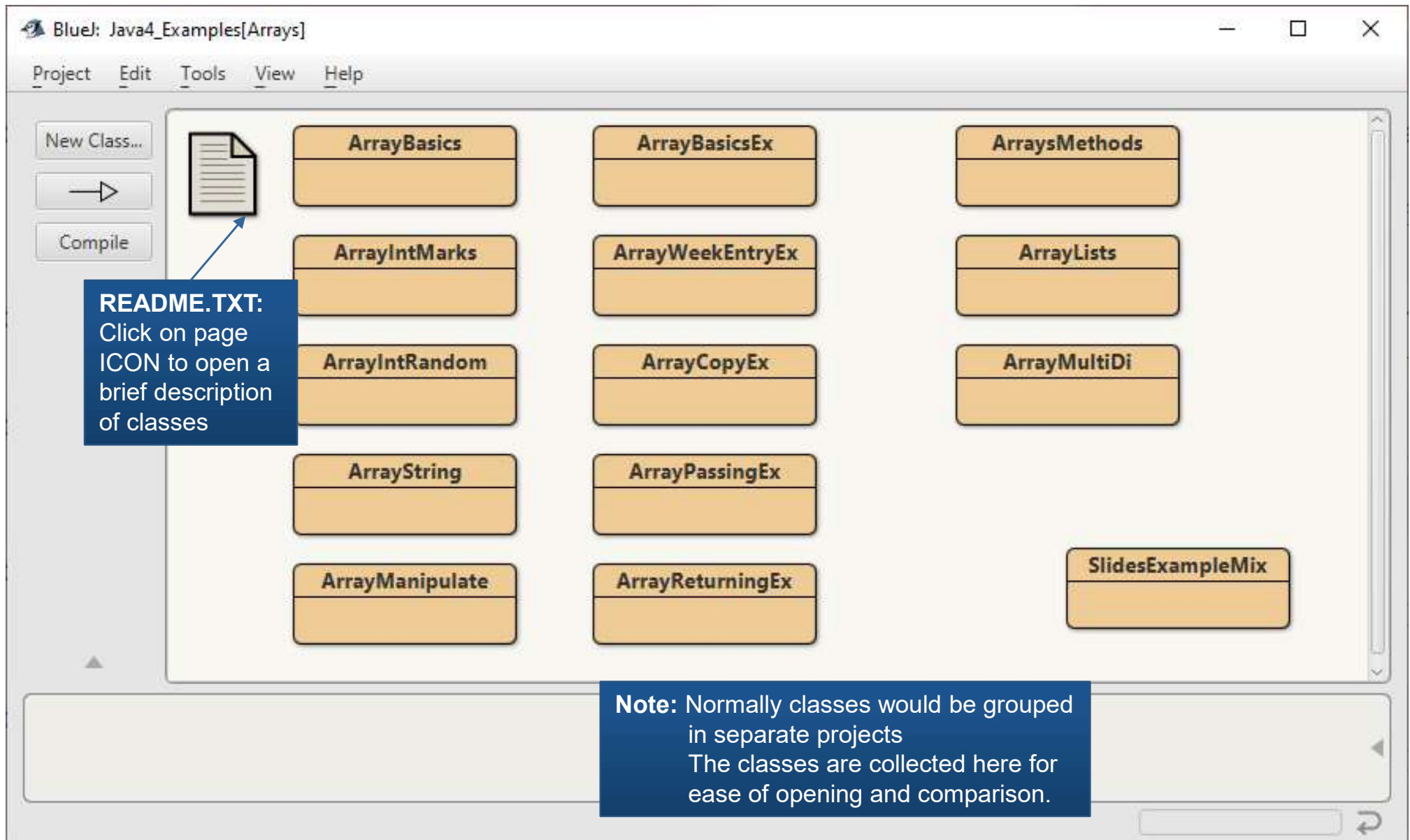
```
{  
    // Declare and initialise    order is [Machine][Sensor][Reading]  
    double[][][] readings = {    // a 3-by-4-by-2  
        { // machine 0  
            {21.5, 24.5}, {20.5, 23.1}, {24.4, 24.2}, {22.7, 20.6} },  
        { // machine 2  
            {31.2, 34.5}, {32.5, 34.1}, {36.8, 38.4}, {30.5, 31.6} },  
        { // machine 3  
            { 0.0,  0.0}, { 0.0,  0.0}, { 0.0,  0.0}, {24.0, 29.9} },  
    };  
}
```

See  
**ArrayMultiDi**  
for an  
alternative:

Declare,  
create and  
assign.

# BlueJ: Java4\_Examples[Arrays]

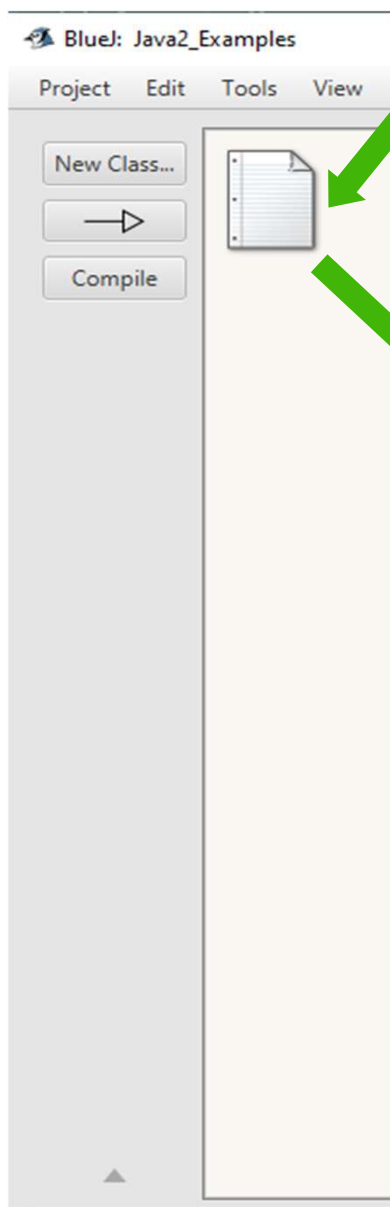
Examples discussed in this section: Go Explore!



# BlueJ: Java4\_Examples

Examples discussed in this section: Go Explore!

**README.TXT:** Click on page ICON to open a brief description



## Various Array Examples

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| ArrayBasics     | Basic examples of array declaration, creation and initialisation |
| ArrayIntMarks   | Demonstrate an integer array                                     |
| ArrayIntRandom  | Demonstrate an integer array and random number generation        |
| ArrayString     | Demonstrate arrays of String references                          |
| ArrayManipulate | Explore Java arrays manipulations.                               |

## Solutions to Exercises

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| ArrayBasicsEx    | Basic exercises for array declaration, creation and initialisation |
| ArrayCopyEx      | Array copy exercise                                                |
| ArrayWeekEntryEx | Exercise: What does weekEntry contain after various operations     |
| ArrayPassingEx   | Exercise: Passing an array to a method                             |
| ArrayReturningEx | Exercise: Returning an array from a method.                        |

## class Arrays and ArrayList

|               |                                                                    |
|---------------|--------------------------------------------------------------------|
| ArraysMethods | Demonstrate the class Arrays methods using an array of type double |
| ArrayLists    | Demonstrate the class ArrayList capabilities and methods           |

## Multidimensional arrays

|                  |                                                                |
|------------------|----------------------------------------------------------------|
| ArrayMultiDi     | Demonstrate Multidimensional arrays both regular and irregular |
| SlidesExampleMix | A wild mixture of examples from Java slides on Arrays          |

# Oracle (Sun) Java Tutorials and Finding help while programming

---

- In general, there is a lot of help [www.oracle.com/technetwork/java](http://www.oracle.com/technetwork/java)
- Java Tutorials are a good set of tutorials on various aspects of Java  
<http://download.oracle.com/javase/tutorial/>
  - [Learning the Java Language](#) > [Language Basics](#) > [Arrays](#)  
<http://download.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>
- Searching from a Browser (using Google, Yahoo, etc):
  - Search for “java arrays oracle” should return the tutorial on arrays as match:
  - Search for “java 23 *className*”  
normally returns as first match the details for the class named *className*  
**java 23** refers to the current version “Java Platform Standard Edition 23”
  - Search for:      **“java 23 Scanner”**      will give details about the Scanner class  
                    **“java 23 Arrays”**        for the Arrays class